

Interactive Problems

Hidden Integer

```
#include

using namespace std;
const int maxX = 1e9;

int main(){
    string result;
    int l = 0, r = maxX + 1;
    while(l + 1 < r){
        int m = l + (r-l) / 2;

        cout << "? " << m << endl;
        cin >> result;

        if(result == "YES") l = m;
        else r = m;
    }

    cout << "! " << l+1 << endl;
}
```

Hidden Permutation

```
#include

using namespace std;
const int maxN = 1001;

int N, ans[maxN];
string result;
vector inv;

int binsearch(int i, int l, int r){
    if(l == r) return l-1;

    int m = l + (r-l) / 2;
    cout << "? " << i << " " << inv[m-1] << endl;
    cin >> result;

    if(result == "YES") return binsearch(i, l, m);
    else return binsearch(i, m+1, r);
}

int main(){
    cin >> N;

    if(N == 1){
        cout << "! 1" << endl;
        return 0;
    }
}
```

```

    inv.push_back(1);
    for(int i = 2; i <= N; i++){
        int pos = binsearch(i, 1, i);
        inv.emplace(inv.begin() + pos, i);
    }

    for(int i = 1; i <= N; i++)
        ans[inv[i-1]] = i;

    cout << "! ";
    for(int i = 1; i < N; i++)
        cout << ans[i] << " ";
    cout << ans[N] << endl;
}

```

Permuted Binary Strings

```

#include

using namespace std;
const int maxSize = 1024;

int N, ans[maxSize];
string response;

int main(){
    cin >> N;
    for(int i = 1; (1<<(i-1)) < N; i++){
        cout << "? ";
        for(int j = 0; j < N; j++)
            cout << (bool) (j & (1<<(i-1)));
        cout << endl;

        cin >> response;
        for(int j = 0; j < N; j++)
            if(response[j] == '1')
                ans[j] += (1<<(i-1));
    }

    cout << "! ";
    for(int i = 0; i < N; i++)
        cout << ans[i] + 1 << (i == N-1 ? "" : " ");
    cout << endl;
}

```

Colored Chairs

```

#include

using namespace std;

```

```
int N;

char query(int i){
    cout << "? " << ((i - 1) % N) + 1 << endl;
    char c;
    cin >> c;
    return c;
}

int main(){
    cin >> N;

    int l = 1, r = N+1;
    char l_color = query(1);
    char r_color = l_color;

    while(l+1 < r){
        int m = (l + r) / 2;
        char m_color = query(m);

        bool left_good = ((m - l) % 2 == 0) ^ (m_color == l_color);

        if(left_good){
            r = m;
            r_color = m_color;
        } else {
            l = m;
            l_color = m_color;
        }
    }

    cout << "! " << l << endl;
}
```
